

UNIVERSITÉ PROTESTANTE AU CONGO

FACULTÉ DES SCIENCES INFORMATIQUES

LICENCE 2 – LMD

Monographie

MedBox AI

Boîtier de diagnostic médical embarqué basé sur l'intelligence artificielle

Réalisé par :

BIDIKI Enock

Année académique 2025 – 2026

Résumé

Ce projet de monographie s'inscrit dans le cadre de l'initiation à la démarche scientifique et à l'ingénierie embarquée au sein de la Faculté des Sciences Informatiques. Il porte sur la conception d'un système médical embarqué complet, nommé MedBox AI, capable de mesurer automatiquement plusieurs constantes vitales d'un patient et de proposer localement un diagnostic parmi dix-neuf profils physiologiques et pathologiques, à l'aide d'un modèle d'intelligence artificielle embarqué sur un microcontrôleur. L'approche suivie combine la constitution d'un jeu de données médicales, l'entraînement d'un réseau de neurones de classification, la conversion de ce modèle pour une exécution locale sur microcontrôleur, la conception d'un système électronique intégrant plusieurs capteurs biométriques et un module d'identification par carte à puce sans contact, ainsi que le développement d'une chaîne logicielle complémentaire assurant la remontée opportuniste des résultats vers une plateforme web de suivi. Les résultats obtenus démontrent la faisabilité d'un diagnostic médical préliminaire entièrement autonome et sans connexion réseau, complété par un historique consultable, tout en révélant plusieurs limites techniques inhérentes à un prototype construit avec des composants grand public. Ce travail illustre ainsi la capacité de l'étudiant à mobiliser des compétences pluridisciplinaires, allant de l'intelligence artificielle à l'électronique embarquée en passant par le développement web, pour concevoir un système fonctionnel répondant à une problématique concrète.

Table des matières

Résumé	1
1 Introduction générale	4
1.1 Contexte	4
1.2 Problématique	4
1.3 Objectifs	5
1.3.1 Objectif général	5
1.3.2 Objectifs spécifiques	5
1.4 Méthodologie adoptée	5
1.5 Structure du document	6
2 Cadre théorique et technologique	7
2.1 Définitions des concepts clés	7
2.2 Présentation des outils ou technologies utilisés	8
3 Réalisation du projet	10
3.1 Analyse des besoins	10
3.2 Architecture de la solution	10
3.3 Étapes de mise en œuvre	12
3.4 Résultats obtenus	14
4 Difficultés rencontrées et limites	17
4.1 Difficultés rencontrées	17
4.2 Limites de la solution	18
5 Conclusion générale	20
5.1 Récapitulatif	20
5.2 Apports du projet	20
5.3 Perspectives	21
6 Bibliographie et Webographie	22
Bibliographie	22

7	Annexes	24
7.1	Annexe 1 – Extrait du code d’inférence embarquée	24
7.2	Annexe 2 – Extrait de la correction de lecture NFC (authentification) . . .	25
7.3	Annexe 3 – Architecture du modèle (Python/Keras)	25
7.4	Annexe 4 – Liste des 19 classes diagnostiquées	26
7.5	Annexe 5 – Format du protocole série et exemple de ligne transmise	26
7.6	Annexe 6 – Intégration Chart.js (PHP vers JavaScript)	26
7.7	Annexe 7 – Synthèse des choix logiciels retenus	28

Chapitre 1

Introduction générale

1.1 Contexte

L'accès à un premier diagnostic médical fiable et rapide reste un enjeu majeur, en particulier dans les contextes où le personnel soignant est peu disponible ou éloigné (zones rurales, postes de secours, dépistage de masse). Les objets connectés couplés à l'intelligence artificielle embarquée, communément désignée par le terme *TinyML*, offrent aujourd'hui une réponse crédible à cette problématique : ils permettent de réaliser une analyse médicale préliminaire directement sur un microcontrôleur, sans connexion internet ni serveur distant. C'est dans cette optique que s'inscrit le projet MedBox AI, un boîtier autonome construit autour d'un microcontrôleur ESP32, capable de mesurer plusieurs constantes vitales d'un patient et d'exécuter localement un modèle d'intelligence artificielle pour proposer un diagnostic.

1.2 Problématique

La problématique centrale du projet peut être formulée ainsi : comment concevoir un système embarqué, autonome et peu coûteux, capable de mesurer automatiquement les constantes vitales d'un patient et d'en déduire, localement et en temps réel, un diagnostic parmi un ensemble de profils physiologiques et pathologiques courants, tout en restant réalisable avec des composants électroniques grand public et des bibliothèques logicielles ouvertes ?

Sous-questions techniques

- Quelles grandeurs physiologiques mesurer, et avec quels capteurs, pour obtenir une donnée exploitable par un modèle d'intelligence artificielle ?
- Comment entraîner un modèle de classification suffisamment précis tout en restant assez léger pour tenir dans la mémoire limitée d'un microcontrôleur ?

- Comment garantir la fiabilité des mesures issues de capteurs bas coût, sujettes au bruit et aux artefacts de mouvement ?
- Comment organiser le déroulement logiciel (acquisition, traitement, inférence, affichage) de manière robuste, malgré les contraintes matérielles du microcontrôleur ?

1.3 Objectifs

1.3.1 Objectif général

Concevoir et réaliser un prototype de boîtier médical embarqué, autonome et fonctionnel, capable de mesurer les constantes vitales d'un patient et de fournir localement un diagnostic à l'aide d'un modèle d'intelligence artificielle, en mobilisant les compétences acquises en programmation, en électronique embarquée et en apprentissage automatique.

1.3.2 Objectifs spécifiques

- Concevoir un boîtier matériel intégrant les capteurs nécessaires à la mesure de quatre grandeurs : âge, fréquence cardiaque, saturation en oxygène et température corporelle.
- Constituer un jeu de données représentatif de dix-neuf profils physiologiques et pathologiques, puis entraîner un modèle de classification multiclasse.
- Convertir ce modèle au format TensorFlow Lite et l'embarquer sur le microcontrôleur pour une inférence entièrement locale.
- Développer une machine à états gérant l'identification du patient, l'acquisition des capteurs, l'inférence de l'intelligence artificielle et l'affichage du résultat.
- Identifier, documenter et corriger les dysfonctionnements rencontrés lors de l'intégration matérielle et logicielle.
- Évaluer les performances et les limites du système à travers des tests, tant en simulation qu'avec le matériel réel.

1.4 Méthodologie adoptée

Le travail s'est déroulé en plusieurs étapes successives. L'étudiant a d'abord mené une analyse du besoin et une recherche des composants électroniques adaptés. Il a ensuite procédé à la constitution d'un jeu de données et à l'entraînement d'un modèle d'intelligence artificielle sous Python, avant de convertir ce modèle pour une exécution embarquée. La phase suivante a consisté en l'implémentation pratique du firmware sur microcontrôleur ESP32, organisée en modules logiciels indépendants (capteurs, identification, intelligence artificielle, affichage). Des tests répétés, en simulation puis avec les capteurs physiques, ont

permis de détecter et corriger plusieurs dysfonctionnements. Tout au long du processus, l'étudiant a été encadré par un enseignant référent.

1.5 Structure du document

Ce document est structuré en cinq chapitres principaux. Le premier chapitre, *Introduction générale*, présente le contexte, la problématique, les objectifs, la méthodologie et l'organisation du travail. Le deuxième chapitre, *Cadre théorique et technologique*, définit les concepts clés mobilisés et décrit les outils et technologies utilisés. Le troisième chapitre, *Réalisation du projet*, présente les étapes concrètes du développement de la solution. Le quatrième chapitre, *Difficultés rencontrées et limites*, expose les contraintes techniques rencontrées et les limites observées du prototype. Enfin, le cinquième chapitre, *Conclusion générale*, récapitule le travail effectué, présente les apports du projet et propose des perspectives d'amélioration.

Chapitre 2

Cadre théorique et technologique

2.1 Définitions des concepts clés

Intelligence artificielle embarquée (TinyML). Sous-domaine de l'apprentissage automatique consistant à déployer des modèles d'intelligence artificielle directement sur des microcontrôleurs à ressources très limitées (quelques kilo-octets à quelques méga-octets de mémoire), sans dépendre d'un serveur distant. Cette approche est au cœur de MedBox AI, dont le modèle de diagnostic s'exécute entièrement sur l'ESP32.

Réseau de neurones de type Perceptron Multicouche (MLP). Modèle d'apprentissage automatique composé de plusieurs couches de neurones artificiels interconnectés, capable d'apprendre à classer des données à partir d'exemples. Dans ce projet, un MLP est utilisé pour classer les mesures d'un patient parmi dix-neuf profils physiologiques et pathologiques.

Normalisation Z-score. Technique de mise à l'échelle des données consistant à soustraire la moyenne et diviser par l'écart-type de chaque variable, afin que toutes les variables d'entrée du modèle aient une échelle comparable. Cette étape est indispensable pour la stabilité de l'apprentissage d'un réseau de neurones.

Quantification de modèle. Processus de compression d'un modèle d'intelligence artificielle consistant à réduire la précision numérique de ses paramètres, afin de diminuer sa taille en mémoire et d'accélérer son exécution sur un matériel aux ressources limitées.

Communication I2C. Protocole de communication série permettant à plusieurs composants électroniques (capteurs, écran, module de communication) de dialoguer avec un microcontrôleur sur un même bus, en utilisant seulement deux fils.

Photopléthysmographie. Technique de mesure optique consistant à éclairer la peau (généralement le doigt) avec une lumière infrarouge et rouge, puis à analyser la lumière réfléchie pour en déduire la fréquence cardiaque et la saturation en oxygène du sang.

Communication en champ proche (NFC). Technologie de communication sans fil à très courte portée, utilisée dans ce projet pour lire l'âge d'un patient stocké sur un

badge, via un module dédié.

Communication série. Mode de transmission de données caractère par caractère sur une liaison filaire (ici, le câble USB reliant le microcontrôleur à un ordinateur), utilisé dans ce projet pour faire remonter les résultats de diagnostic vers une base de données, sans recourir à une connexion réseau.

Architecture client-serveur et pages dynamiques (PHP). Modèle dans lequel un serveur génère du contenu (ici des pages web) à la demande, en interrogeant une base de données, avant de l'envoyer au navigateur du client. Contrairement à une page statique, une page dynamique affiche toujours des données à jour.

Opérations CRUD. Acronyme désignant les quatre opérations fondamentales de manipulation de données dans une application : création (*Create*), lecture (*Read*), mise à jour (*Update*) et suppression (*Delete*). Ce modèle structure la conception de la plateforme web du projet.

JSON (*JavaScript Object Notation*). Format de données textuel léger, utilisé pour transmettre des informations structurées entre un serveur (PHP) et un script exécuté dans le navigateur (JavaScript).

2.2 Présentation des outils ou technologies utilisés

- **ESP32** : microcontrôleur central du projet, choisi pour sa puissance de calcul suffisante, son faible coût et sa capacité à exécuter des modèles d'intelligence artificielle allégés. Il pilote l'ensemble des périphériques et exécute le firmware complet.
- **MAX30102** (bibliothèque SparkFun) : capteur optique de photopléthysmographie permettant de mesurer la fréquence cardiaque et la saturation en oxygène. Choisi pour son faible coût et sa large documentation communautaire.
- **MLX90614** (bibliothèque Adafruit) : capteur infrarouge sans contact permettant de mesurer la température corporelle du patient sans contact direct, pour des raisons d'hygiène.
- **PN532** (bibliothèque Adafruit) : module de lecture NFC utilisé pour identifier le patient et extraire son âge depuis un badge.
- **Écran OLED SSD1306** : afficheur utilisé pour présenter les mesures en temps réel puis le diagnostic final à l'utilisateur.
- **Python, avec les bibliothèques pandas, scikit-learn et TensorFlow/Keras** : environnement utilisé pour préparer le jeu de données, entraîner le modèle de classification et le convertir au format embarqué.
- **TensorFlow Lite for Microcontrollers** (via les bibliothèques `tflm_esp32` et `eloquent_tinym1`) : cadriciel permettant d'exécuter le modèle d'intelligence artificielle quantifié directement sur l'ESP32.
- **Arduino IDE / C++** : environnement de développement utilisé pour écrire,

compiler et téléverser le firmware embarqué sur l'ESP32.

- **Microsoft Excel** : utilisé pour la constitution et l'organisation du jeu de données de 9500 échantillons ayant servi à l'entraînement du modèle.
- **Python (bibliothèques pyserial et pymysql)** : utilisé pour développer le programme relais (« pont ») assurant la lecture du port série de l'ESP32 et l'enregistrement des mesures dans la base de données.
- **MySQL** : système de gestion de base de données relationnelle utilisé pour stocker de manière persistante les patients et leurs mesures successives.
- **PHP** : langage de programmation côté serveur utilisé pour développer les quatre pages de la plateforme web (tableau de bord, fiche patient, liste des patients, file de finalisation), en générant du HTML à jour à chaque consultation.
- **Chart.js (bibliothèque JavaScript, hébergée localement)** : utilisée pour dessiner les courbes d'évolution des constantes vitales d'un patient dans son historique, sans nécessiter de connexion internet.
- **HTML/CSS** : utilisés pour la structure et la mise en forme visuelle de la plateforme web, avec une direction graphique inspirée des interfaces de dispositifs médicaux.

Chapitre 3

Réalisation du projet

3.1 Analyse des besoins

Le système devait répondre aux besoins fonctionnels suivants :

- Identifier un patient de manière simple et rapide, sans saisie manuelle.
- Mesurer automatiquement, sans intervention d'un professionnel de santé, quatre grandeurs physiologiques : âge, fréquence cardiaque, saturation en oxygène et température corporelle.
- Traiter ces données localement pour produire un diagnostic parmi dix-neuf profils possibles, sans dépendre d'une connexion internet.
- Afficher un résultat clair et lisible à l'utilisateur.
- Conserver un historique des mesures de chaque patient, consultable ultérieurement, afin de permettre un suivi dans le temps plutôt qu'une simple mesure ponctuelle.

Ces besoins fonctionnels s'accompagnaient de contraintes non fonctionnelles fortes, propres à l'embarqué : mémoire vive très limitée (quelques centaines de kilo-octets), nécessité d'un fonctionnement autonome (sans serveur), et budget matériel restreint imposant l'usage de capteurs grand public plutôt que de dispositifs médicaux certifiés.

3.2 Architecture de la solution

Le système MedBox AI repose sur une architecture globale en trois niveaux : le dispositif embarqué, qui réalise le diagnostic de manière autonome ; un programme relais, qui fait remonter les mesures vers une base de données lorsque les conditions le permettent ; et une plateforme web, qui permet de consulter l'historique des patients. Cette organisation garantit qu'aucun de ces trois niveaux n'est indispensable au fonctionnement des autres : le diagnostic reste possible même sans remontée de données, et la remontée reste possible même sans consultation web immédiate.

FIGURE 3.1 – Schéma de l’architecture globale de MedBox AI (dispositif, pont série, base de données, plateforme web)

Le dispositif embarqué

Le boîtier MedBox AI repose sur un microcontrôleur ESP32 connecté en bus I2C à l’ensemble des périphériques. L’architecture logicielle est organisée en modules indépendants, chacun responsable d’un sous-système :

- `config.h` : définition des états de la machine, de la structure de données patient, des constantes de normalisation et de calibration.
- `nfc_module.h` : gestion de l’identification du patient via le module PN532.
- `sensors_module.h` : gestion de l’acquisition des mesures physiologiques (MAX30102, MLX90614).
- `ia_inference.h` : normalisation des données et exécution de l’inférence TensorFlow Lite Micro.
- `display_module.h` : gestion de l’affichage sur l’écran OLED.

Le fonctionnement global est piloté par une **machine à états finis**, qui structure le cycle de vie complet d’une mesure patient :

1. `STATE_WAIT_NFC` : attente du scan d’un badge NFC, permettant d’extraire l’âge du patient.
2. `STATE_READ_SENSORS` : acquisition de la température (détection de présence), puis de la fréquence cardiaque et de la saturation en oxygène.
3. `STATE_SCALING` : normalisation Z-score des quatre variables mesurées.
4. `STATE_INFERENCE` : exécution de l’inférence TinyML locale.
5. `STATE_SHOW_RESULT` : affichage du diagnostic pendant dix secondes, avant réinitialisation pour un nouveau patient.

La remontée des données par liaison série

Le dispositif ne communique jamais directement avec la base de données. La remontée des mesures s’effectue par liaison série (le câble USB reliant déjà le dispositif à un ordinateur), plutôt que par le réseau WiFi, pour rester cohérente avec l’objectif d’autonomie hors ligne du dispositif : aucune configuration réseau n’est nécessaire, et les données transmises restent lisibles directement pour faciliter le débogage.

Un programme relais, appelé « pont » et écrit en Python, écoute en continu le port série. À chaque mesure, le dispositif imprime une ligne de texte au format suivant, les champs étant séparés par des points-virgules et préfixés par le mot-clé `MEDBOX` ; :

```
MEDBOX;uid;age;bpm;spo2;temperature;confiance;classe
```

Ce préfixe permet au pont de distinguer les lignes de mesure des nombreux autres messages que le dispositif imprime par ailleurs (démarrage, étapes de mesure) : seules les lignes commençant exactement par **MEDBOX** ; sont traitées, les autres étant ignorées.

Cette remontée est qualifiée d'**opportuniste** : elle n'a lieu que si un câble est branché et qu'un pont est actif à l'écoute, mais le diagnostic réalisé par le dispositif n'en dépend jamais. La base de données ne joue ainsi qu'un rôle de mémoire optionnelle, jamais celui d'un maillon indispensable au fonctionnement médical du système.

La plateforme web de suivi

Une plateforme web, développée en PHP et connectée à une base de données MySQL, permet de consulter l'historique des mesures. Elle se compose de quatre pages couvrant les opérations CRUD (création, lecture, mise à jour, suppression) : un tableau de bord (statistiques et dernières mesures), une fiche patient (historique et courbes d'évolution), une liste des patients (avec suppression), et une file de finalisation (complétion des patients créés automatiquement à partir d'un badge inconnu). La création d'un patient n'est pas assurée par une page web, mais déclenchée automatiquement dès la réception d'une mesure associée à un badge inconnu.

Les courbes d'évolution des constantes vitales sont dessinées avec la bibliothèque JavaScript Chart.js, hébergée localement sur le serveur pour un fonctionnement hors ligne. Les données, lues en base par PHP, sont transmises au script JavaScript via la fonction `json_encode`, qui convertit un tableau PHP en un format directement exploitable par Chart.js. Un contrôle affiche les courbes uniquement à partir de deux mesures disponibles, une mesure isolée ne pouvant traduire aucune tendance.

Le style visuel de la plateforme s'inspire des interfaces de dispositifs médicaux (fond clair, palette bleu profond et sarcelle, cartes à coins arrondis), avec un codage couleur à quatre niveaux (normal, alerte, sévère, critique) permettant de repérer une situation grave en un coup d'œil — une information que l'écran OLED du dispositif, plus contraint, ne pouvait pas transmettre.

3.3 Étapes de mise en œuvre

Phase de conception

Le projet a débuté par la définition des dix-neuf profils physiologiques et pathologiques à reconnaître (états normaux selon l'âge, variations physiologiques légères, et pathologies de sévérité croissante telles que tachycardie, bradycardie, hypoxie, fièvre ou choc). Les seuils retenus pour chaque classe (plages de fréquence cardiaque, de saturation en oxygène et de température selon l'âge) ont été construits à partir de tables de référence clinique

reconnues, notamment le Manuel MSD, les directives *Pediatric Advanced Life Support* de l’American Heart Association pour les seuils pédiatriques, ainsi que les repères cliniques de détresse et de choc publiés par l’Organisation mondiale de la santé.

Un jeu de données synthétique de 9500 échantillons a ensuite été constitué, parfaitement équilibré entre les dix-neuf classes (500 échantillons par classe), avec pour chaque échantillon quatre variables d’entrée (âge, fréquence cardiaque, saturation en oxygène, température) et une étiquette de classe. Le recours à des données synthétiques plutôt qu’à des données patients réelles se justifie à la fois sur le plan éthique et sur le plan pratique : les principes relatifs aux données de l’Organisation mondiale de la santé rappellent qu’aucune donnée réelle de patient ne doit être utilisée sans infrastructure de protection et d’anonymisation adaptée, hors de portée d’un projet étudiant. Sur le plan scientifique, la littérature reconnaît par ailleurs l’usage de données synthétiques comme une pratique validée en santé numérique, en particulier pour l’éducation, la formation et le développement de logiciels de santé, ainsi que pour tester des algorithmes et générer des hypothèses avant toute manipulation de données réelles.

Phase d’implémentation

Entraînement du modèle d’intelligence artificielle. Un réseau de neurones de type Perceptron Multicouche a été entraîné sous Python/Keras, avec l’architecture suivante : une couche d’entrée à 4 neurones, une première couche cachée de 32 neurones (activation ReLU), une couche de régularisation (Dropout à 10 %), une seconde couche cachée de 16 neurones (activation ReLU), et une couche de sortie de 19 neurones (activation Softmax). L’entraînement a été réalisé avec l’optimiseur Adam, une fonction de perte de type *categorical crossentropy*, sur 50 itérations complètes (*epochs*) et des lots de 32 échantillons, sur un ensemble d’entraînement représentant 80 % du jeu de données (les 20 % restants formant l’ensemble de test).

Conversion pour l’embarqué. Le modèle entraîné a ensuite été converti au format TensorFlow Lite avec quantification par défaut, afin de réduire sa taille et d’accélérer son exécution sur microcontrôleur. Le fichier obtenu a été transformé en tableau d’octets C++, directement intégrable dans le firmware. Sur l’ESP32, l’inférence est exécutée avec une zone mémoire de seulement 8 kilo-octets de SRAM, illustrant la légèreté du modèle final.

Développement du firmware embarqué. Le firmware a été développé en C++ sous Arduino IDE, avec la structure modulaire décrite en section 3.2. Un mode de simulation a été ajouté à deux niveaux du système : un mode de simulation NFC (badge simulé par une valeur d’âge fixe dans le code) et un mode de saisie manuelle des constantes vitales

via le moniteur série, permettant de tester les dix-neuf classes de sortie du modèle sans avoir à reproduire physiquement chaque état pathologique avec les capteurs.

Développement du pont série et de la base de données. Un programme relais Python, exploitant les bibliothèques `pyserial` et `pymysql`, a été développé pour écouter le port série, filtrer les lignes de mesure valides et les enregistrer dans une base de données MySQL. Une logique d'enrôlement automatique des patients a été implémentée : un badge inconnu déclenche la création d'un nouveau patient au statut « en attente de finalisation », complété ultérieurement via la plateforme web.

Développement de la plateforme web. Les quatre pages de la plateforme (tableau de bord, fiche patient, liste des patients, file de finalisation) ont été développées en PHP, avec une interrogation directe de la base MySQL à chaque consultation. L'intégration de la bibliothèque Chart.js, associée à la fonction PHP `json_encode` pour transmettre les données au script JavaScript, a permis d'afficher les courbes d'évolution des constantes vitales de chaque patient.

Phase de tests

Les tests ont été menés à deux niveaux. D'une part, le modèle a été évalué automatiquement sur l'ensemble de test Python (1900 échantillons non vus pendant l'entraînement). D'autre part, le prototype matériel complet a été testé de manière itérative, ce qui a permis de mettre en évidence plusieurs dysfonctionnements (détaillés au chapitre 4) et d'orienter les corrections successives apportées au firmware, notamment sur la fiabilité de la mesure cardiaque et sur la lecture des badges NFC.

3.4 Résultats obtenus

Performance du modèle d'intelligence artificielle

Après entraînement, le modèle atteint une précision de **95,58 %** sur l'ensemble de test. Cette précision est quasiment intégralement préservée après conversion et quantification au format TensorFlow Lite, confirmant que la compression du modèle n'a pas dégradé significativement ses performances.

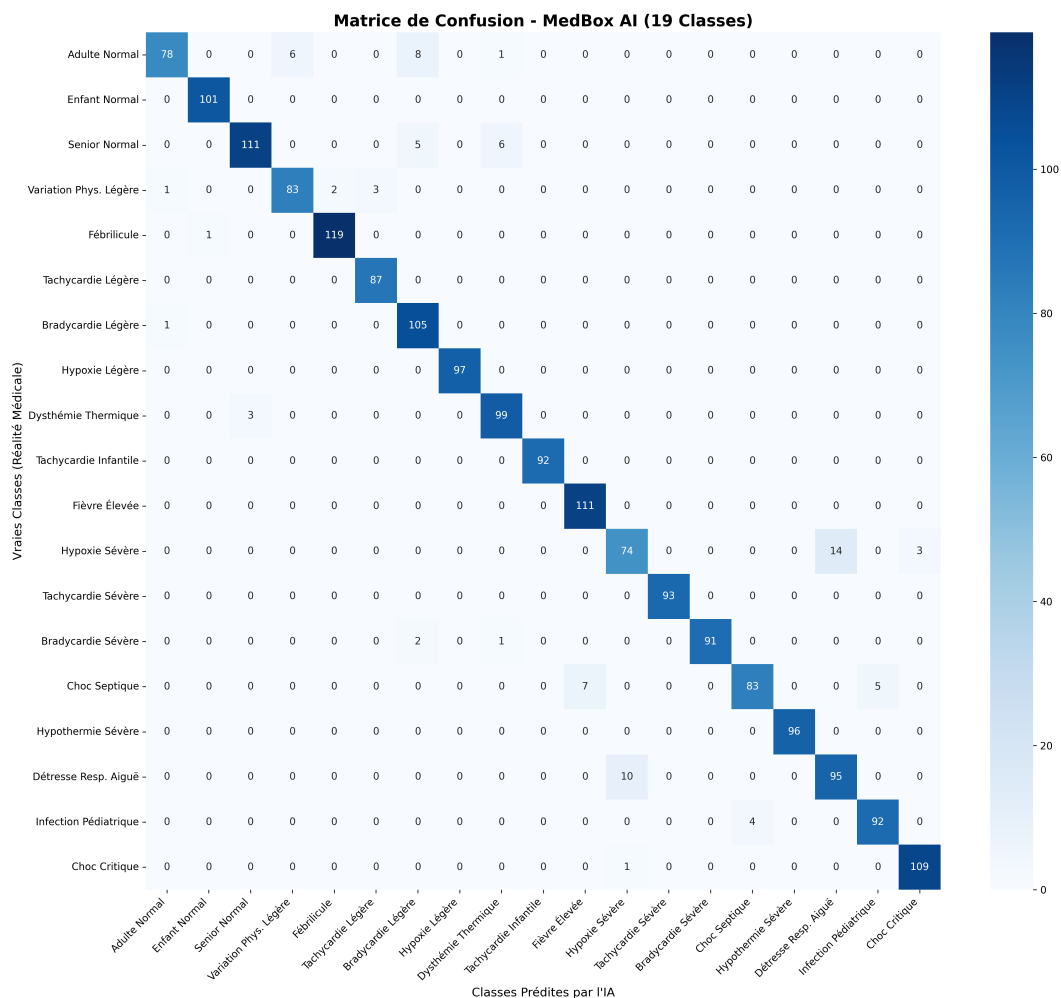


FIGURE 3.2 – Matrice de confusion du modèle MedBox AI sur les 19 classes

L'analyse de la matrice de confusion (figure 3.2) montre que la quasi-totalité des classes sont très bien reconnues. Les rares confusions observées concernent presque exclusivement des classes physiologiquement proches, par exemple *Adulte Normal* confondu occasionnellement avec *Variation Physiologique Légère* ou *Tachycardie Légère*, ou encore *Hypoxie Sévère* confondue avec *Infection Pédiatrique Grave* — des états graves partageant des profils de constantes vitales proches. Ces confusions restent limitées et concentrées sur des cas cliniquement voisins, ce qui confirme la pertinence du modèle pour un usage de premier tri, tout en soulignant qu'il ne saurait remplacer un diagnostic médical complet.

Résultats du prototype matériel

Une fois le firmware corrigé (voir chapitre 4), le boîtier fonctionne de bout en bout : lecture d'un badge NFC, mesure de la température par calibration empirique, mesure de la fréquence cardiaque et de la saturation en oxygène par cycles successifs, puis inférence et affichage du diagnostic. La calibration du capteur de température, réalisée à partir de huit mesures comparatives avec un thermomètre de référence, a permis de déterminer un

offset moyen de correction de $+3,98$ °C (écart-type des écarts individuels de $0,83$ °C), rendant les températures transmises au modèle cohérentes avec la température corporelle réelle. Concernant la mesure cardiaque, des essais répétés sur un même sujet au repos ont révélé une variabilité significative des valeurs retenues (de 83 à 166 battements par minute selon les essais), reflétant les limites du capteur bas coût utilisé plutôt qu'un défaut de la chaîne logicielle de traitement.

Résultats de la remontée de données et de la plateforme web

La chaîne complète de remontée des données a été validée : chaque diagnostic produit par le dispositif est imprimé sur le port série au format attendu, capté par le pont Python, puis enregistré dans la base MySQL avec application correcte de la logique d'enrôlement (rattachement à un patient existant ou création automatique d'un nouveau patient pour un badge inconnu). La plateforme web permet de consulter cet historique : le tableau de bord affiche les dernières mesures, la fiche patient présente les courbes d'évolution des constantes vitales dès que deux mesures ou plus sont disponibles, et le codage couleur à quatre niveaux de gravité permet une lecture rapide de la situation d'un patient. Le caractère opportuniste de cette remontée a également été vérifié : le dispositif continue de produire des diagnostics normalement lorsque le pont n'est pas actif, confirmant que la base de données reste une fonctionnalité complémentaire et non une dépendance du système de diagnostic.

Chapitre 4

Difficultés rencontrées et limites

4.1 Difficultés rencontrées

Plusieurs dysfonctionnements techniques ont été identifiés et corrigés au cours du développement du prototype. Ce chapitre les documente de manière transparente, dans un souci d'honnêteté scientifique.

Authentification manquante en lecture NFC. La fonction de lecture des badges Mifare Classic appelait la lecture d'un bloc mémoire sans authentification préalable du secteur, alors qu'une carte Mifare Classic exige cette authentification avant toute lecture. La lecture échouait donc silencieusement et le système retombait systématiquement sur une valeur d'âge de secours. Correction apportée : ajout de l'authentification par clé par défaut avant chaque lecture du bloc concerné.

Incohérence de format entre écriture et lecture NFC. Le script d'écriture initial stockait l'âge en texte, alors que la logique de lecture interprétait le premier octet comme une valeur numérique brute, produisant des âges erronés en sortie. Correction apportée : alignement du format d'écriture sur le format brut attendu en lecture.

Calibration insuffisante du capteur de température. Le capteur infrarouge sans contact mesure la température de peau, systématiquement inférieure de plusieurs degrés à la température corporelle centrale. Sans correction, l'entrée transmise à l'intelligence artificielle restait proche de 32-33 °C, provoquant une classification systématique en « hypothermie sévère ». Correction apportée : calibration empirique à partir de huit mesures comparatives, avec un offset moyen de +3,98 °C appliqué uniquement à la valeur transmise à l'IA et à l'affichage.

Absence de retour visuel après le scan du badge. Après un scan réussi, l'écran restait figé sur l'écran d'attente jusqu'à la fin complète de la mesure des capteurs. Correction

apportée : ajout d'un écran de confirmation intermédiaire (âge lu et signal sonore).

Crash mémoire lors de l'agrandissement de la fenêtre de capture cardiaque.

Une tentative d'agrandir la fenêtre de capture du capteur cardiaque (de 100 à 400 échantillons), pour tenter de stabiliser le calcul du rythme cardiaque, a provoqué un incident critique (*Guru Meditation Error*). La cause identifiée est que la bibliothèque de calcul utilisée repose sur des tableaux de travail internes de taille fixe, codée en dur à 100 échantillons, indépendamment du paramètre transmis. Dépasser cette limite provoque une écriture hors des zones mémoire allouées. Correction apportée : retour à une fenêtre de 100 échantillons (contrainte non modifiable sans réécrire la bibliothèque), compensée par la multiplication du nombre de cycles de mesure indépendants (passé de 1 à 5).

Contention du bus I2C pendant la capture cardiaque. L'écran OLED et le capteur cardiaque partageant le même bus I2C, le rafraîchissement de l'écran pendant la capture bloquait le bus 20 à 30 millisecondes à chaque appel, perturbant la régularité temporelle du signal nécessaire au calcul fiable du rythme cardiaque. Correction apportée : suppression des mises à jour d'écran pendant la capture, l'affichage n'étant désormais rafraîchi qu'une seule fois au début de chaque cycle de mesure.

Instabilité persistante des mesures cardiaques. Malgré les corrections précédentes, des valeurs physiologiquement improbables ont continué à apparaître ponctuellement, en particulier lors de micro-mouvements du doigt. Plusieurs corrections cumulatives ont été apportées : renforcement du lissage matériel, multiplication des cycles de mesure indépendants, filtrage des valeurs hors plage physiologique plausible (fréquence cardiaque entre 40 et 200 battements par minute, saturation entre 70 et 100 %), calcul de la médiane des cycles valides plutôt qu'une moyenne, et ajout d'un garde-fou détectant les cas de doublement de fréquence (phénomène harmonique connu de l'algorithme de calcul), avec préférence donnée à la fréquence fondamentale la plus basse.

Séparation du firmware d'écriture des badges. L'écriture et la lecture des badges NFC ne pouvant cohabiter dans un même programme exécuté par le microcontrôleur, un programme indépendant a dû être développé spécifiquement pour la préparation des badges, nécessitant un changement de programme téléversé à chaque modification de badge.

4.2 Limites de la solution

- **Précision de la mesure de température.** L'offset de calibration repose sur seulement huit mesures comparatives réalisées sur un nombre restreint de sujets.

La marge d'erreur réaliste à attendre en usage reste supérieure à celle d'un thermomètre médical certifié; cette calibration est indicative et n'a pas été validée cliniquement.

- **Fiabilité du capteur cardiaque.** Même après filtrage, cycles multiples et calcul par médiane, des écarts de mesure significatifs ont été observés sur une même personne au repos. Cette variabilité reflète une limite réelle du capteur bas coût utilisé et de l'algorithme de démonstration associé, qui n'intègrent pas les techniques de rejet de mouvement des oxymètres de pouls commerciaux certifiés.
- **Absence de validation clinique.** Ni les capteurs, ni le modèle de classification, ni les seuils de décision utilisés n'ont été validés par du personnel médical qualifié ni comparés à des dispositifs certifiés. MedBox AI reste un prototype éducatif et ne doit en aucun cas être utilisé comme outil de diagnostic réel.
- **Dépendance au mode de simulation.** Le paramètre de simulation NFC doit être explicitement désactivé pour un usage avec de véritables badges. Un oubli à ce niveau fausserait silencieusement les résultats, sans message d'erreur explicite à l'utilisateur.
- **Couverture de test incomplète du modèle.** La validation des dix-neuf classes de sortie repose majoritairement sur le mode de saisie manuelle des constantes vitales, faute de pouvoir reproduire physiquement chaque état critique avec les capteurs disponibles. La performance du modèle sur des données de capteurs réelles pour ces classes critiques reste donc partiellement non vérifiée.
- **Contrainte matérielle non résolue.** La fenêtre de capture du signal cardiaque, imposée par la bibliothèque de calcul utilisée, n'a pas pu être allongée sans provoquer un crash mémoire. Une fenêtre plus longue améliorerait probablement la stabilité du calcul, mais nécessiterait de réécrire l'algorithme de traitement du signal, ce qui n'a pas été réalisé dans le cadre de ce projet.

Chapitre 5

Conclusion générale

5.1 Récapitulatif

Ce projet a porté sur la conception d'un système médical embarqué complet, MedBox AI, combinant capteurs biométriques, identification par badge NFC, intelligence artificielle embarquée, remontée de données par liaison série et plateforme web de suivi. La démarche suivie a consisté à constituer un jeu de données de 9500 échantillons répartis en dix-neuf profils physiologiques et pathologiques, à entraîner un modèle de classification atteignant 95,58 % de précision, puis à convertir et intégrer ce modèle sur un microcontrôleur ESP32 via TensorFlow Lite Micro. Le firmware embarqué, organisé autour d'une machine à états, a été développé, testé et corrigé de manière itérative pour aboutir à un prototype fonctionnel. Ce dispositif a ensuite été complété par un programme relais assurant l'enregistrement opportuniste des mesures dans une base de données, ainsi que par une plateforme web permettant de consulter l'historique et l'évolution de chaque patient.

5.2 Apports du projet

Ce travail a permis à l'étudiant de mobiliser et de développer des compétences pluridisciplinaires : préparation et exploitation d'un jeu de données, entraînement et évaluation d'un modèle d'apprentissage automatique, conversion de modèles pour l'embarqué, programmation en C++ sur microcontrôleur, intégration de capteurs électroniques hétérogènes sur un même bus de communication, démarche rigoureuse de diagnostic et de correction de dysfonctionnements matériels et logiciels, ainsi que conception d'une chaîne logicielle complète allant du capteur jusqu'à une interface web de suivi (communication série, base de données relationnelle, développement web côté serveur et visualisation de données). Le projet a également permis d'appréhender concrètement l'écart, souvent sous-estimé, entre un modèle d'intelligence artificielle performant en simulation et son comportement une fois confronté aux contraintes réelles d'un système embarqué et de capteurs

physiques imparfaits.

5.3 Perspectives

Plusieurs perspectives d'amélioration se dégagent de ce travail. L'enrichissement du jeu de données avec des mesures réelles, et non uniquement synthétiques, permettrait de renforcer la robustesse du modèle en conditions réelles. Le remplacement ou la réécriture de l'algorithme de traitement du signal cardiaque permettrait de dépasser la contrainte actuelle de la fenêtre de capture. Une calibration du capteur de température sur un échantillon plus large de sujets renforcerait la fiabilité de cette mesure. La remontée de données actuellement filaire (liaison série) pourrait évoluer vers une transmission WiFi optionnelle, tout en conservant le principe d'un diagnostic qui ne dépend jamais du réseau. Enfin, le développement d'une application mobile pour le personnel soignant, ainsi qu'une validation clinique par du personnel médical qualifié, constitueraient des étapes naturelles vers un dispositif plus abouti, dans la continuité d'un futur mémoire ou projet de fin de cycle.

Chapitre 6

Bibliographie et Webographie

Bibliographie

Justification du recours à des données synthétiques

- Gonzales, A., Guruswamy, G., Smith, S. R. *Synthetic data in health care : A narrative review*. PLOS Digital Health. 2023 ;2(1) :e0000082. doi : 10.1371/journal.pdig.0000082. Disponible sur : <https://pmc.ncbi.nlm.nih.gov/articles/PMC9931305/>, consulté en juillet 2026.
- Kokosi, T., Harron, K. *Synthetic data in medical research*. BMJ Medicine. 2022 ;1(1) :e000167. doi : 10.1136/bmjmed-2022-000167. Disponible sur : <https://bmjmedicine.bmj.com/content/1/1/e000167>, consulté en juillet 2026.
- Organisation mondiale de la santé (OMS). *Principes relatifs aux données de l'OMS*. Disponible sur : <https://data.who.int/fr/about/data/who-data-principles>, consulté en juillet 2026.

Seuils médicaux de référence (constantes vitales, 19 classes)

- MSD Manuals. *Manuel MSD – Édition professionnelle, tables de référence cliniques*. Disponible sur : <https://www.msmanuals.com/professional/pages-with-widgets/tables>, consulté en juillet 2026.
- Medtronic. *Normal vital signs – reference infosheet (pulse oximetry, respiratory rate monitoring)*. Disponible sur : <https://www.medtronic.com/content/dam/medtronic-wide/public/western-europe/products/patient-monitoring/pulse-oximetry-resparray-monitor-normal-vitals-infosheet.pdf>, consulté en juillet 2026.
- American Heart Association. *Pediatric Advanced Life Support (PALS) Guidelines – Normal Pediatric Vital Signs*. Disponible sur : <https://papers.ualgary.ca/paediatrics/assets/normal-pediatric-vital-signs.pdf>, consulté en juillet 2026.

- Organisation mondiale de la santé (OMS). *Reference Card for WHO Emergency Unit*. Disponible sur : <https://www.who.int/docs/default-source/documents/emergency-care/reference-card-for-who-emergency-unit-form-general.pdf>, consulté en juillet 2026.

Remarque : les dates de consultation ci-dessus sont indicatives (juillet 2026, période de rédaction du présent document) ; l'étudiant est invité à les ajuster si une consultation ultérieure a lieu, et à compléter cette liste avec toute autre source effectivement utilisée durant le projet (cours, documentation technique des bibliothèques logicielles, etc.).

Chapitre 7

Annexes

7.1 Annexe 1 – Extrait du code d'inférence embarquée

```
1 float applyZScoreScaling(float rawValue, float mean, float stdDev
  ) {
2     if (stdDev == 0.0f) return 0.0f;
3     return (rawValue - mean) / stdDev;
4 }
5
6 int runLocalInference(const PatientData &patient) {
7     input->data.f[0] = applyZScoreScaling(patient.age, MEAN_AGE,
8         STD_AGE);
9     input->data.f[1] = applyZScoreScaling(patient.bpm, MEAN_BPM,
10        STD_BPM);
11     input->data.f[2] = applyZScoreScaling(patient.spo2, MEAN_SPO2
12        , STD_SPO2);
13     input->data.f[3] = applyZScoreScaling(patient.temperature,
14        MEAN_TEMP, STD_TEMP);
15
16     TfLiteStatus invoke_status = interpreter->Invoke();
17
18     int predictedClass = 0;
19     float maxConfidence = -1.0;
20     for (int i = 0; i < NUM_CLASSES; i++) {
21         float confidence = output->data.f[i];
22         if (confidence > maxConfidence) {
23             maxConfidence = confidence;
24             predictedClass = i;
25         }
26     }
```

```

21     }
22 }
23 return predictedClass;
24 }

```

7.2 Annexe 2 – Extrait de la correction de lecture NFC (authentification)

```

1 uint8_t keyA[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };
2 if (!nfc.mifareclassic_AuthenticateBlock(uid, uidLength, 4, 0,
3     keyA)) {
4     Serial.println("Echec authentication bloc 4 (lecture
5         impossible).");
6 } else if (nfc.mifareclassic_ReadDataBlock(4, data)) {
7     readSuccess = true;
8     if (data[0] > 0 && data[0] <= 120) {
9         extractedAge = (float)data[0];
10    }
11 }

```

7.3 Annexe 3 – Architecture du modèle (Python/-Keras)

```

1 model = Sequential([
2     Dense(32, activation='relu', input_shape=(4,)),
3     Dropout(0.1),
4     Dense(16, activation='relu'),
5     Dense(19, activation='softmax')
6 ])
7
8 model.compile(
9     optimizer='adam',
10    loss='categorical_crossentropy',
11    metrics=['accuracy']
12 )

```

7.4 Annexe 4 – Liste des 19 classes diagnostiquées

Idx	Classe	Idx	Classe
0	Adulte Normal	10	Fièvre Élevée (Infection)
1	Enfant Normal	11	Hypoxie Sévère (Détsresse)
2	Senior Normal	12	Tachycardie Sévère
3	Variation Physiologique Légère	13	Bradycardie Sévère (Syncope)
4	Fébricule (Alerte)	14	Choc Septique
5	Tachycardie Légère (Stress)	15	Hypothermie Sévère
6	Bradycardie Légère	16	Détsresse Respiratoire Aiguë
7	Hypoxie Légère	17	Infection Pédiatrique Grave
8	Dysthémie Thermique (Froid)	18	Choc Critique (Instable)
9	Tachycardie Infantile		

TABLE 7.1 – Correspondance entre les indices de sortie du modèle et les 19 classes

7.5 Annexe 5 – Format du protocole série et exemple de ligne transmise

```

1 // Format imprime par le dispositif a chaque mesure :
2 // MEDBOX;uid;age;bpm;spo2;temperature;confiance;classe
3
4 // Exemple reel :
5 // MEDBOX;A3F2C19B;8;95.0;98.0;36.38;0.9234;1

```

Listing 7.1 – Boucle d'écoute simplifiée du pont Python

```

1 # ouvrir le port serie et la connexion a la base
2 while True:
3     ligne = port_serie.readline().decode().strip()
4     if ligne.startswith("MEDBOX;"):
5         champs = ligne.split(";")
6         uid, age, bpm, spo2, temp, confiance, classe = champs[1:]
7         enregistrer_mesure(uid, age, bpm, spo2, temp, confiance,
8                             classe)
9     # sinon, la ligne est ignoree (message de debogage du
10    dispositif)

```

7.6 Annexe 6 – Intégration Chart.js (PHP vers JavaScript)

```
1 // Cote PHP : lecture des mesures depuis la base, puis encodage
   JSON
2 $serieBpm = [78, 95, 110];
3 $jsonBpm = json_encode($serieBpm); // "[78,95,110]"
```

Listing 7.2 – Creation de la courbe cote JavaScript

```
1 new Chart(document.getElementById('chartBpm'), {
2     type: 'line',
3     data: {
4         labels: labels,
5         datasets: [{
6             data: dataBpm,
7             borderColor: '#2E7D8E',
8             fill: true
9         }]
10    }
11 });
```

7.7 Annexe 7 – Synthèse des choix logiciels retenus

Choix	Justification
Calibration par offset additif simple	Absence de moyens et de temps pour collecter un jeu de données de calibration suffisamment large pour une régression multi-variable.
Mesure en plusieurs cycles + médiane	La bibliothèque de calcul du rythme cardiaque impose une taille de buffer fixe non modifiable ; la multiplication des cycles compense cette contrainte.
Mode de saisie manuelle des constantes vitales	Permet de valider les dix-neuf classes de sortie du modèle indépendamment de la fiabilité des capteurs physiques.
Firmware d'écriture NFC séparé	Évite d'alourdir le firmware embarqué final avec du code de préparation de badges non nécessaire en usage courant.
Filtrage par plage physiologique plausible	Empêche qu'une valeur aberrante isolée ne pollue le résultat final, sans nécessiter d'algorithme de traitement du signal plus complexe.
Liaison série plutôt que WiFi	Cohérence avec l'objectif d'autonomie hors ligne du dispositif ; absence de configuration réseau ; débogage plus simple.
Remontée opportuniste des données	Garantit que le diagnostic ne dépend jamais de la disponibilité de la base de données ou du pont.
Chart.js hébergé localement	Permet à la plateforme web de fonctionner sans connexion internet, cohérent avec le reste du système.

TABLE 7.2 – Choix logiciels retenus et justifications